EduHaskell project report

Dylan Heydon-Matterface

Trinity 2027

Contents

0.1 Introduction 1
0.1.1 Motivation 1
0.2 Parsing 2
0.2.1 Framework 2
0.2.2 Grammar 2

0.1 Introduction

0.1.1 Motivation

The functional programming paradigm offers a unique and interesting way of reasoning about program code. Oxford introduces its students to functional programming through teaching Haskell, but this is not a perfect solution for a beginner. Consider, for example, the following[1]:

```
ghci> xs = [1,2,3]
ghci> max xs

<interactive>:3:1: error: [GHC-39999]
    • No instance for ‘Show ([Integer] -> [Integer])’
        arising from a use of ‘print’
        (maybe you haven’t applied a function to enough arguments?)
    • In a stmt of an interactive GHCi command: print it
```

In order to understand this output, we must first know how the GHCi REPL evaluates expressions and prints their values to the terminal, so that we realise it is trying to print a function. This may be unnecessarily confusing for the novice functional programmer. We might instead desire something like the following:

```
> xs = [1,2,3]
> max xs

! Type error
! ‘max xs’ has type [Integer] -> [Integer], which cannot be printed.
```

[1]Example taken from Bernard's original project proposal.

In order to obtain such simplicity in error messages, we necessarily must remove some of the complexity that Haskell offers, such as type classes, so that our language is more readily able to identify that our programmer has simply made a mistake, rather than assuming they wish to somehow print a function to the screen.

0.2 Parsing

0.2.1 Framework

As previously mentioned, EduHaskell is written in Scala. In order to have a parser compatible with the interpreter, the parser is implemented using ScalaLR[2].

0.2.2 Grammar

At the top level, every EduHaskell program is a sequence of actions:

```
program: Unit = (NL)? (anAction NL)+ ENDOFFILE { () }
```

Each of these actions is one of `expr`, `equation`, `typeDef`, or `guardedEquation`. `expr` is subdivided into many different types of valid expression, for example:

```
caseExpression: Expr = 'case' query:expression 'of' alt
                           { Case($query, List($alt)) }
                     | 'case' query:expression 'of' '{' alts '}'
                           { Case($query, $alts) }
```

In the syntax of ScalaLR, productions such as the above are of the form

```
x: A = y1 { z1 } | y2 { z2 } | ...
```

where x is the name of this production, A is its Scala type, the y_i are rules for matching a sequence of tokens, and the z_i are the resulting Scala objects arising should the corresponding rule match the sequence of input tokens. Using the example above, we apply production `caseExpression` whenever we see a sequence of input tokens of the form `case e of { a1 ; a2 ; ... }`, where e is a valid expression and each a_i is e.g. `Some(x) -> x`.

`equation` parses equational definitions, such as `f x = x + 3`. `typeDef` parses type annotations, such as `f :: Integer -> Integer`. `guardedEquation` parses definitions which involve various guarded cases, for example:

```
abs x | x >= 0 = x
      | x < 0  = -x
```

Example 0.2.1. The input expression

```
abs x = if x >= 0 then x else -x
```

[2] `https://github.com/sufrin/scalalr`

is parsed to produce the following abstract syntax tree:

